AUTHOR

NAME : SNEHAPRIYA THIAGARAJAN
ROLL NO : 23f3001500
EMAIL : 23F3001500 SNEHAPRIYA THIAGARAJAN

DESCRIPTION

This is a Vehicle Parking App with two types of logins: one for users, who can register, reserve parking spots, mark their vehicle as occupied or available, and view their parking history with timestamps and cost; and another for admins, who can create and manage parking lots and spots, monitor which spots are occupied or available, view and manage registered users, and generate summary charts for effective parking lot management.

TECHNOLOGIES USED

Python – The core programming language used to develop the backend logic of the app.
HTML – For defining the structure and content of the webpage
Bootstrap – To make the frontend aesthetically appealing and easily navigable .
Jinja – To fill web pages with various contents like quiz questions and scores.
SQLite – Database to store quizzes, users, and scores.
Flask – To run the backend and connect everything together.
Chart.js – For interactive and visual admin analytics.

DATABASE SCHEMA DESIGN

1) User
    a) id: Integer, Primary Key
    b) username: String, unique, cannot have null value
    c) password: String, cannot have null value
    d) vehicle_number: String
2) ParkingLot
    a) id: Integer, Primary Key
    b) name: String, cannot have null value
    c) address: String , cannot have null value
    d) pincode: String , cannot have null value
    e) price: Float , cannot have null value

f) max_spots: Integer , cannot have null value

3) ParkingSpot
    a) id: Integer, Primary Key
    b) lot_id: Integer , Foreign Key → ParkingLot Id , cannot have null value
    c) spot_number: Integer , cannot have null value
    d) status: String , cannot have null value (A- available , O - Occupied)
4) Reservation
    a) id: Integer, Primary Key
    b) spot_id: Integer , Foreign Key → ParkingLot Id , cannot have null value
    c) user_id: Integer , Foreign Key → User Id , cannot have null value
    d) start_time: DateTime , default = current time
    e) end_time: DateTime , can be null
    f) cost: Float , can be null

In general, all classes use id as the primary key. Foreign key relationships are established between Reservations and ParkingSpots ,Reservations and Users ,ParkingSpots and ParkingLot .

ARCHITECTURE AND FEATURES

Architecture

As per the recommended flash app structure

1. Contains the database schema and all routes that connect the database to the frontend templates.
2. Templates folder is used to serve the html files
3. Static folder contains the css
4. The instance folder has the database defined .
5. env contains the required python libraries used to build the application

Features

The login page opens to :

- ★ **User Login**: Registered users can log in using their username and password to access their user dashboard.
- ★ **Admin Login:** Admin logs in using pre-defined credentials to access the admin panel.
- ★ **User Registration**: New users can register with a unique username, password, and vehicle numbe**r**.

User Dashboard:

After logging in, users land on the homepage, where they can

- ★ View a list of available parking lots.
- ★ Reserve a parking spot from a selected lot (auto-allocation of the first free spot).
- ★ Mark the spot as occupied when parked and mark it available again after use.
- ★ View start and end timestamps for each parking session.
- ★ Automatically calculate and view the parking cost based on time.
- ★ See their active and past reservations.

Admin Panel:

Admins have full control over the system and can:

- ★ **Create a new Parking Lot**: Define name, address, pincode, price, and number of spots.
- ★ **View Spot Status**: See if each spot is **available** or **occupied**, along with user details.
- ★ **Edit/Delete Parking Lots**: Admin can update lot details or remove a lot if all spots are free.
- ★ **View Registered Users**: See all users and delete them if needed.
- ★ **View Summary Charts**: Visual analytics using **Chart.js** showing parking lot occupancy and availability.

Video Link :

IITM MAD 1 project.mp4